A

🔥 Angular Top 30 Interview Q&A

1. What is Angular?

Angular is a **TypeScript-based front-end framework** by Google used to build **single-page applications (SPA)** with features like routing, dependency injection, and reactive programming.

2. Angular vs AngularJS?

AngularJS	Angular
JavaScript	TypeScript
MVC	Component-based
Two-way binding heavy	Optimized change detection
Slower	Faster

3. What are Components?

A component is a **building block of Angular UI**, consisting of:

HTML (template)

CSS (styles)

TypeScript (logic)

4. What is a Module?

Modules (NgModule) organize the app into cohesive blocks.

Example:

```
@NgModule({
declarations: [],
imports: [],
providers: [],
bootstrap: []
})
```

5. What is Data Binding?

Ways to bind data between UI and TS:

Interpolation {{ }}

Property binding [value]

Event binding (click)

Two-way binding [(ngModel)]

6. What is Dependency Injection?

A design pattern where dependencies are **injected instead of created manually**.

7. What are Services?

Reusable logic classes used across components.

8. What is RxJS?

Reactive programming library using Observables.

9. What is Observable?

A stream of async data (API calls, events).

10. Observable vs Promise?

Observable	Promise
Observables are for streams of events over time	A Promise can only handle one event
It is cancel-able/retry-able and supports operators such as map,filter,reduce etc.	Not easy to cancel and it returns a single value
stream data in multiple pipelines	having one pipeline
Provides chaining and subscription to handle complex applications	Uses only .then() clause
Subscribe method is used for error handling which makes centralized and predictable error handling	Push errors to the child promises

11. What is Angular CLI?

Command-line tool to create/manage Angular apps:

ng new app
ng generate component

12. What is Routing?

Navigation between components using:

RouterModule.forRoot(routes)

13. What is Lazy Loading?

Loading modules only when needed → improves performance.

14. What are Directives?

Modify DOM behavior.

Types:

Structural (*ngIf, *ngFor)

Attribute (ngClass, ngStyle)

15. What is Pipe?

Transforms data in template:

{{ date | date }}

16. Pure vs Impure Pipe?

Pure → runs when input changes

Impure → runs every change detection

17. What is Change Detection?

Angular updates UI when data changes.

Strategies:

Default

OnPush (performance optimized)

18. What is Zone.js?

Tracks async operations and triggers change detection.

19. What is ViewChild?

Access DOM or child component:

@ViewChild('ref') element;

20. What is Content Projection?

Using <ng-content> to pass content into components.

21. Lifecycle Hooks?

Common ones:

ngOnInit

ngOnChanges

ngAfterViewInit

ngOnDestroy

22. What is Reactive Forms?

Form handling using FormControl & FormGroup.

23. Template-driven vs Reactive Forms?

Template	Reactive
Simple	Complex
HTML driven	TS driven

24. What is HTTPClient?

Used for API calls:

this.http.get()

25. What is Interceptor?

Intercept HTTP requests globally (auth, logging).

26. What is Guards?

Protect routes.

Types:

CanActivate

CanDeactivate

Resolve

27. What is TrackBy in ngFor?

Optimizes rendering:

*ngFor="let item of list; trackBy: trackById"

28. What is AOT vs JIT?

AOT	JIT
Compile before run	Compile in browser
Faster	Slower

29. What is SSR (Angular Universal)?

Server-side rendering for SEO and performance.

30. What are Signals (Latest Angular)?

New reactive state system (Angular 16+):

```
const count = signal(0);
```

🔥 1. RxJS Operators (map, switchMap, mergeMap)

✅ map

Transforms each emitted value.

```
this.http.get('/users')
.pipe(map((res: any) => res.data));
```

👉 Use when: You just need to **transform data**

✅ switchMap (MOST ASKED)

Cancels previous request and switches to new one.

```
this.searchInput.valueChanges
.pipe(
switchMap(value => this.http.get(`/api?q=${value}`))
)
.subscribe();
```

👉 Use when:

API search

Auto-suggestions

Avoid duplicate calls

👉 Key point:

✅ Cancels previous request

✅ Prevents race conditions

✅ mergeMap

Runs multiple observables in parallel.

```
from([1,2,3])
.pipe(mergeMap(id => this.http.get(`/api/${id}`)))
.subscribe();
```

👉 Use when:

Multiple independent API calls

No need to cancel previous ones

💡 **Interview Tip:**

switchMap → cancel previous

mergeMap → parallel

concatMap → sequential

🔥 2. OnPush Change Detection

✅ What is it?

Angular updates UI **only when input reference changes** instead of checking everything.

```
@Component({
changeDetection: ChangeDetectionStrategy.OnPush
})
```

✅ When it triggers:

Input reference changes

Event occurs

Observable emits (async pipe)

✅ Why use it?

✓ Better performance
✓ Avoid unnecessary re-renders

🔥 3. Lazy Loading Architecture

✅ What is it?

Load modules **only when user navigates**

✅ Example:

```
{
path: 'admin',
loadChildren: () => import('./admin/admin.module')
.then(m => m.AdminModule)
}
```

✅ Benefits:

✓ Faster initial load
✓ Better scalability
✓ Code splitting

💡 Real Interview Point:

Used in large enterprise apps

Combine with route guards

🔥 4. Interceptors + Auth Handling

✅ What is Interceptor?

Intercept HTTP requests globally.

✅ Example:

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
intercept(req: HttpRequest<any>, next: HttpHandler) {
const token = localStorage.getItem('token');

const cloned = req.clone({
headers: req.headers.set('Authorization', `Bearer ${token}`)
});

return next.handle(cloned);
}
}
```

✅ Use Cases:

✔ Add JWT token
✔ Logging
✔ Error handling

💡 Advanced Interview Point:

Handle **401 errors + refresh token**

🔥 5. Reactive Forms Validation

✅ Example:

```
this.form = this.fb.group({
email: ['', [Validators.required, Validators.email]],
password: ['', [Validators.required, Validators.minLength(6)]]
});
```

✅ Template:

```
<input formControlName="email">
<div *ngIf="form.get('email')?.invalid">
Invalid Email
</div>
```

✅ Custom Validator:

```
function passwordMatch(control: AbstractControl) {
return control.value === '123' ? null : { mismatch: true };
}
```

💡 Interview Points:

Reactive forms = scalable + testable

Dynamic form building possible

🔥 6. State Management (NgRx Basics)

✅ What is NgRx?

State management library based on **Redux pattern**

✅ Core Concepts:

1. Store

Single source of truth

2. Actions

Events that trigger state change

```
export const loadUsers = createAction('[User] Load');
```

3. Reducer

Pure function to update state

```
const reducer = createReducer(
initialState,
on(loadUsers, state => ({ ...state, loading: true }))
);
```

4. Effects

Handle async operations (API calls)

```
loadUsers$ = createEffect(() =>
this.actions$.pipe(
ofType(loadUsers),
switchMap(() => this.api.getUsers())
)
);
```

5. Selector

Read data from store

```
this.store.select(selectUsers);
```

✅ **When to Use NgRx?**

✓ Large applications
✓ Complex state sharing
✓ Multiple API interactions

🔥 1. Typeahead Search (RxJS + API) – MOST ASKED

✅ Problem

Create a search box that:

Calls API while typing

Avoids multiple calls

Cancels previous request

✅ Solution

```
searchControl = new FormControl();

ngOnInit() {
this.searchControl.valueChanges.pipe(
debounceTime(300),
distinctUntilChanged(),
switchMap(value => this.http.get(`/api/search?q=${value}`))
).subscribe(res => {
this.results = res;
});
}
```

✅ Key Points to Say

debounceTime → reduce API calls

switchMap → cancel previous request

distinctUntilChanged → avoid duplicate calls

🔥 2. Auto Save Form (Reactive Form + RxJS)

✅ Problem

Auto-save form data when user stops typing.

✅ Solution

```
this.form.valueChanges.pipe(
debounceTime(500),
switchMap(value => this.http.post('/api/save', value))
).subscribe();
```

✅ Interview Insight

👉 Use switchMap to prevent overlapping save calls

🔥 3. Dependent Dropdown (Country → State)

✅ Problem

When country changes → load states from API

✅ Solution

```
this.form.get('country')?.valueChanges.pipe(
switchMap(country =>
this.http.get(`/api/states?country=${country}`))
).subscribe(states => {
this.states = states;
});
```

✅ Bonus Tip

Reset state field:

```
this.form.get('state')?.reset();
```

🔥 4. Parallel API Calls (Dashboard Data)

✅ Problem

Load multiple APIs at once for dashboard

✅ Solution

```
forkJoin({
users: this.http.get('/api/users'),
orders: this.http.get('/api/orders')
}).subscribe(res => {
this.users = res.users;
this.orders = res.orders;
});
```

✅ Why forkJoin?

✓ Waits for all APIs
✓ Returns combined result

🔥 5. Retry Failed API (Resilience)

✅ Problem

Retry API call if it fails

✅ Solution

```
this.http.get('/api/data').pipe(
retry(3),
catchError(err => {
console.error(err);
return of([]);
})
).subscribe();
```

✅ Interview Point

Improves reliability

Combine with interceptor

🔥 6. Form Validation with API (Async Validator)

✅ Problem

Check if username already exists

✅ Solution

```
usernameValidator(control: AbstractControl) {
return this.http.get(`/api/check?user=${control.value}`).pipe(
map((res: any) => res.exists ? { taken: true } : null)
);
}

this.form = this.fb.group({
username: ['', null, [this.usernameValidator.bind(this)]]
});
```

✅ Key Point

👉 Async validation = real-world use case

🔥 7. Loading Spinner with API Calls

✅ Problem

Show loader while API is running

✅ Solution

```
this.loading = true;

this.http.get('/api/data').pipe(
finalize(() => this.loading = false)
).subscribe(res => {
this.data = res;
});
```

✅ Interview Tip

Use finalize() for cleanup

🔥 8. Cancel API on Component Destroy

✅ Problem

Prevent memory leaks

✅ Solution

```
destroy$ = new Subject();

ngOnInit() {
this.http.get('/api/data')
.pipe(takeUntil(this.destroy$))
.subscribe();
}

ngOnDestroy() {
this.destroy$.next();
this.destroy$.complete();
}
```

✅ Key Point

👉 Must for Angular apps (memory optimization)

🔥 9. Combine Multiple Inputs (Search Filters)

✅ Problem

Search with multiple filters

✅ Solution

```
combineLatest([
this.category.valueChanges,
this.search.valueChanges
]).pipe(
debounceTime(300),
switchMap(([cat, search]) =>
this.http.get(`/api?cat=${cat}&q=${search}`)
)
).subscribe();
```

✅ Why Important?

✓ Real-world filtering
✓ Tests RxJS knowledge deeply

🔥 10. File Upload with Progress

✅ Problem

Upload file with progress bar

✅ Solution

```
upload(file: File) {
const formData = new FormData();
formData.append('file', file);

this.http.post('/api/upload', formData, {
reportProgress: true,
observe: 'events'
}).subscribe(event => {
if (event.type === HttpEventType.UploadProgress) {
this.progress = Math.round(100 * event.loaded / event.total!);
}
});
}
```

🔥 1. Why use switchMap vs mergeMap?

✅ Core Difference

switchMap → Cancels previous request

mergeMap → Runs all requests in parallel

✅ When to use switchMap

✓ Search / autocomplete
✓ Latest data matters
✓ Avoid race conditions

```
this.search.valueChanges.pipe(
switchMap(q => this.http.get(`/api?q=${q}`))
)
```

👉 If user types fast → old requests are cancelled

✅ When to use mergeMap

✓ Multiple independent API calls
✓ All results required

```
from(ids).pipe(
mergeMap(id => this.http.get(`/api/${id}`))
)
```

💡 Interview One-Liner

👉 “I use **switchMap for canceling stale requests** and **mergeMap for parallel execution when all responses are needed**.”

🔥 2. How do you prevent memory leaks in Angular?

✅ Common Causes

Unsubscribed Observables

Long-lived subscriptions

✅ Best Practices

✓ 1. takeUntil (MOST RECOMMENDED)

```
destroy$ = new Subject();

ngOnInit() {
this.http.get('/api')
.pipe(takeUntil(this.destroy$))
.subscribe();
}

ngOnDestroy() {
this.destroy$.next();
this.destroy$.complete();
}
```

✓ 2. Async Pipe (BEST)

```
<div *ngIf="data$ | async as data">
{{ data }}
</div>
```

👉 Angular auto-unsubscribes

✓ 3. One-time APIs

```
this.http.get('/api').subscribe(); // auto completes
```

💡 Interview One-Liner

👉 “I prefer **async pipe and takeUntil pattern** to avoid memory leaks.”

🔥 3. How do you optimize API calls?

✅ Techniques

✅ 1. Debouncing

debounceTime(300)

👉 Avoid unnecessary calls while typing

✅ 2. distinctUntilChanged

distinctUntilChanged()

👉 Prevent duplicate requests

✓ **3. Caching (IMPORTANT)**

shareReplay(1)

```
this.users$ = this.http.get('/api/users').pipe(
shareReplay(1)
);
```

👉 Avoid multiple API hits

✓ **4. Lazy Loading**

👉 Load modules only when needed

✓ **5. Combine APIs**

forkJoin([...])

✓ 6. Use Interceptors

👉 Add headers globally instead of repeating code

💡 Interview One-Liner

👉 “I optimize APIs using **debounce, caching with shareReplay, lazy loading, and avoiding duplicate calls**.”

🔥 4. Error Handling Strategy

✅ 1. Component Level

```
this.http.get('/api').pipe(
catchError(err => {
console.error(err);
return of([]);
})
).subscribe();
```

✅ 2. Global Error Handling (Interceptor)

```
intercept(req, next) {
return next.handle(req).pipe(
catchError((error: HttpErrorResponse) => {
if (error.status === 401) {
// redirect to login
}
return throwError(() => error);
})
);
}
```

✅ **3. Retry Strategy**

retry(2)

✅ **4. User-Friendly Messages**

👉 Don’t expose raw errors to UI

💡 **Advanced Point (Very Impressive)**

✓ Handle **401 with refresh token flow**

✓ Centralized logging (e.g., Azure App Insights)

💡 **Interview One-Liner**

👉 “I handle errors using **catchError locally, interceptors globally, and implement retry + user-friendly messaging**.”

🚀 Final Power Answer (Say This in Interview)

👉

"I use **switchMap to cancel stale requests and mergeMap for parallel calls**, prevent memory leaks using **takeUntil and async pipe**, optimize APIs with **debounce, caching, and lazy loading**, and handle errors using **interceptors, retry strategies, and centralized logging**."